# Studio Nx

JS 118 — tripletTicks()

2026-10-14

# JS 118 — `tripletTicks()` (셋잇단 칸의 tick 길이)

| 항목 | 내용 |
|---|---|
| 파일 | `studionx/frontend/static/js/pages/sequencer-arrange.js` |
| 작업 | S1-06 타임라인: 네 번째 함수 (FR-206 셋잇단 포함) |
| 작성 | Claude (AI), 2026-10-14, 사용자 검토 |
| 브랜치 | `feature/phase1/s1-06-timeline` |
| 다음 | 스냅 단위를 고르는 함수(칸 + 셋잇단 → tick)와 Snap 메뉴 연결을 이어서 진행합니다 |

## 1. 하는 일

`³ Triplet`을 켰을 때 그 칸의 **셋잇단 한 칸 길이**를 tick으로 돌려줍니다. 셋잇단은 **한 칸을 3등분**한 길이입니다 (원래 칸 ÷ 3).

> 셋잇단 한 칸 = (같은 이름 칸의 tick) ÷ 3

| 칸 | 원래 칸 | 셋잇단 한 칸 |
|---|---|---|
| `1/1` | 3840 | 1280 |
| `1/2` | 1920 | 640 |
| `1/4` | 960 | 320 |
| `1/8` | 480 | 160 |
| `1/16` | 240 | 80 |
| `1/32` | 120 | 40 |
| 이름이 틀림(`1/3` 등) | — | 오류(`RangeError`) |

그림으로 보면 `1/4` 칸(960)이 셋잇단일 때 3토막으로 나뉩니다.

```
1/4 칸 (960)
├──────────────┤
├──────┼──────┼──────┤   셋잇단 한 칸 = 320
```

## 2. 코드

```
export function tripletTicks(division) {
  return divisionTicks(division) / 3;
}
```

**최신 문법으로 쓰면** (공부용)

```
export const tripletTicks = (division) => divisionTicks(division) / 3;
```

| 최신 문법 | 실제 코드 | 뜻 |
|---|---|---|
| 화살표 함수 | `function` 선언 | 같은 동작입니다 |

## 3. 왜 이렇게

| 선택 | 이유 |
|---|---|
| 셋잇단 = 원래 칸 ÷ 3 | 2026-10-14 사용자 결정(그림 (나)). 화면 칸 하나가 그대로 3토막 나는 방식이라 직관적입니다 |
| `divisionTicks()`를 그대로 씀 | 칸 이름 검사와 값이 한 곳에서 나옵니다. 없는 이름은 `divisionTicks()`가 던진 오류가 그대로 전해집니다 |
| 3으로 나눔을 함수로 분리 | 셋잇단 규칙(2/3 vs 1/3)이 바뀌어도 고칠 곳이 여기 한 곳입니다 |
| 표준(× 2/3)을 안 씀 | 표준은 셋잇단 칸을 원래 칸보다 길게(3개가 2칸) 잡습니다. 사용자가 (나)를 골랐습니다 |

> 참고: 기존 프로토타입은 '박' 단계가 ÷3, '1/16' 단계가 ×2/3로 서로 달랐습니다. 이 함수로 **모든 칸을 ÷3으로 통일**합니다.

## 4. 시험 (Node에서 직접 실행, 14개 통과)

| 확인 | 결과 |
|---|---|
| `1/1` ~`1/32` 셋잇단이 1280·640·320·160·80·40 tick | ✓ |
| 셋잇단 3개가 원래 한 칸과 같음(`1/4`, `1/16`) | ✓ |
| 셋잇단이 원래 칸의 1/3 | ✓ |
| 없는 칸·빈 문자열·숫자·없음·상속 키 → 오류 | ✓ |